

Pin Cracker

LiangKun Chen and GuanDu Zhao
lc2108@hw.ac.uk and gz2007@hw.ac.uk
Heriot-Watt University - OUC CW2

Abstract

This is a report of the F28HS coursework 2(Heriot-Watt University, 2026). The project consists of two independent projects: The assessed lab and the main part. Our work divide into code and report writing. Our code and video can be found at <https://github.com/KunLiangChen/F28HS-CW2>.

Keywords – C, ASM, Inline code, raspberry Pi

1 Work Division

The work was divided equally between the two group members, with clear responsibilities assigned to leverage each member's strengths.

LiangKun Chen take responsible for Software environment. In detail he finish

- Implement the binary function using inline code and asm.
- Implement the LCD display name and button read
- Implement the advanced hamming distance search function
- Writing the report's software part.

GuanDu Zhao is working on Hardware part. In detail the job include

- Finish wiring
- Write the hardware part of the report
- Wiring the simple way for hamming search
- Take time analysis.

Both members collaborated on debugging, testing the attack against known PINs, recording the demonstration video, and maintaining the GitHub repository.

2 Hardware Configuration

This section details the hardware components utilized in the *Pin Cracker* project and their specific interfacing with the Raspberry Pi 4. The entire system is assembled on a solderless breadboard to facilitate modular testing.

Safety Note: The system primarily operates on 3.3 V logic. Specifically, the button circuit is powered by the 3.3 V rail to prevent over-voltage damage to the GPIO pins.

2.1 Components and Connectivity

2.1.1 Bill of Materials

The following components were used in the assembly:

- **Controller:** Raspberry Pi 4 Model B.
- **Visual Indicators:**
 - Green LED (Status) with 330 Ω resistor.
 - Red LED (Alert) with 330 Ω resistor.
- **Input:** Push button with a 10 k Ω external pull-down resistor.
- **Display:** HD44780-compatible 16x2 LCD and a 10 k Ω potentiometer for contrast control.
- **Interconnects:** Assorted Dupont jumper wires.

2.1.2 GPIO Assignment

All peripherals are mapped to the Raspberry Pi's GPIO header as summarized in Table 1.

Device	GPIO Pin	Function
Green LED (data)	GPIO 26 (pin 37)	Output
Red LED (control)	GPIO 5 (pin 29)	Output
Push button	GPIO 19 (pin 35)	Input
LCD RS	GPIO 25 (pin 22)	Register select
LCD EN	GPIO 24 (pin 18)	Enable
LCD D4	GPIO 23 (pin 16)	Data bit 4
LCD D5	GPIO 10 (pin 19)	Data bit 5
LCD D6	GPIO 27 (pin 13)	Data bit 6
LCD D7	GPIO 22 (pin 15)	Data bit 7

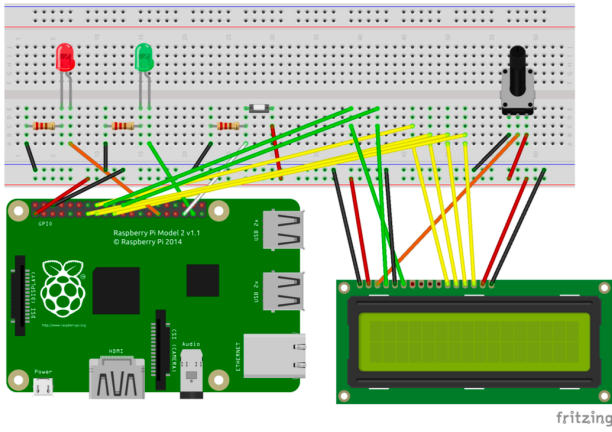
Table 1: GPIO pin assignment for all peripherals.

2.2 Peripheral Wiring Details

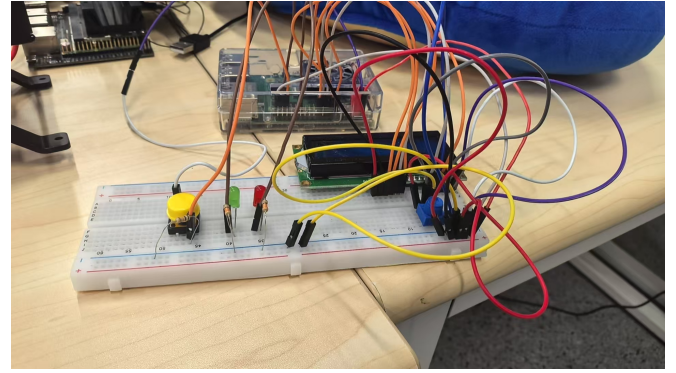
LED Configuration Each LED is connected in series with a 330 Ω current-limiting resistor. The anodes are tied to their respective GPIO pins, while the cathodes are connected to the common Ground (GND). The **Green LED** blinks to acknowledge digit entry, whereas the **Red LED** signals the initiation of a new input sequence.

Button Circuit To ensure signal stability, an external 10 k Ω pull-down resistor is employed. One terminal of the button is connected to the 3.3 V rail; the other is linked to GPIO 19. This configuration ensures that GPIO 19 reads HIGH only when the button is actively pressed.

LCD Interface The LCD operates in **4-bit mode** to optimize GPIO usage. It is powered via the 5 V rail (for the backlight) and 3.3 V for logic. Pin 3 (V0) is connected to the



(a) Fritzing schematic



(b) Physical breadboard implementation

Figure 1: Complete hardware wiring diagram and real-world assembly.

potentiometer's wiper to allow manual contrast adjustment. Detailed pin assignments are provided in Table 2.

Pin	Name	Conn.	Pin	Name	Conn.
1	VSS	GND	9	D2	(unused)
2	VDD	3.3 V	10	D3	(unused)
3	V0	Potentiometer	11	D4	GPIO 23
4	RS	GPIO 25	12	D5	GPIO 10
5	R/W	GND	13	D6	GPIO 27
6	EN	GPIO 24	14	D7	GPIO 22
7	D0	(unused)	15	LED+	3.3V (330Ω)
8	D1	(unused)	16	LED-	GND

Table 2: LCD pinout and connections.

2.3 System Visualization

The complete circuit integration is illustrated below. Figure 1(a) provides the clear logic schematic via Fritzing, while Figure 1(b) showcases the physical implementation on the breadboard.

3 Code Structure

The `pinCrack` application is built using a modular architecture that bridges high-level algorithmic logic with low-level hardware interaction on the Raspberry Pi 4. The system is designed to handle real-time button inputs, manage precise timing via interval timers, and execute high-performance sequence comparisons.

3.1 Main Application Logic

The core logic resides in `cw2.c`, which manages the program lifecycle across two distinct phases:

- **Phase 1: Sequence Input:** Utilizes `initITimer` and `timer_handler` to implement a non-blocking timeout mechanism. The program captures user button presses, using `usleep` for debouncing, to build the `attemptSeq`.
- **Phase 2: PIN Cracking:** The application computes the Hamming distance between the input and the `secret`. It then enters a search loop to identify the PIN.

This is controlled via conditional compilation:

- **TASK4:** A full-space brute force search.
- **Default:** An optimized two-loop search that utilizes `incseq` and combination logic to prune the search space based on the known Hamming distance.

In addition, the code also read a lot of params and they are shown in Table 3

Option	Argument	Function
-d	N/A	Debug: Shows secret/guessed seqs and Hamming distance.
-e	N/A	Exhaustive: Tries all PINs, submits valid ones only.
-l	N/A	LCD test: Prints test message to LCD only.
-u	N/A	Unit test: Compares two seqs (via -s, -r).
-n	<i>seqlen</i>	Sets sequence length .
-m	<i>maxval</i>	Sets max value (range: 1 to <i>maxval</i>).
-S	<i>delay</i>	Sets submit delay (in μ s).
-s	<i>secret</i>	Sets fixed secret sequence for debugging.
-r	<i>reference</i>	Sets reference sequence (skips buttons).

Table 3: Optimized command-line interface for `cw2`.

3.2 Core Functionality Table

The primary functions used within the system are summarized in Table 4.

3.3 Build and Configuration

The Makefile orchestrates the compilation of C and S files. It allows for toggling between the optimized assembly Hamming implementation (`-DHAMM_ASM`) and the full-space search logic (`-DTASK4`), ensuring the system can be tuned for either performance or educational demonstration.

4 Access the Hardware

4.0.1 Hardware Interfacing

The hardware abstraction and driver layer of the `cw2` application are strategically decoupled into two primary components:

Table 4: Key Functions and Descriptions

Function	Description
initITimer	Configures the itimer and signal handlers for user input timeouts.
timehandler	Callback function to set timeout.
hamming	Calculates differences between sequences (ASM or C).
submit_PIN	Simulates an expensive verification check with a mandatory delay.
incseq	Manages sequence increments for arbitrary lengths and digit bases.
blinkN	Provides visual feedback through LED pulses.

lcd-binary.c and lcd-fcts.c.

The lcd-binary.c module serves as the low-level foundation, providing highly optimized, atomic operations for direct register manipulation. By bypassing standard library overhead and utilizing inline assembly, it ensures precise timing for GPIO control which is critical for the BCM2711 peripheral interface.

Conversely, lcd-fcts.c operates at a higher level of abstraction, implementing the complex command sequences and initialization protocols required by the HD44780U LCD controller. This separation of concerns not only facilitates easier debugging of the physical signaling but also enhances the portability of the code across different ARM-based platforms. The core low-level primitives implemented in this layer are detailed below:

pin_mode

Configures the designated GPIO pin as either INPUT or OUTPUT. This function implements a multi-stage selection logic to determine the correct **Function Select Register** based on the pin number. It utilizes **inline ARM assembly** to perform a Read-Modify-Write operation: clearing the existing 3-bit configuration using BIC and setting the new mode using ORR with a calculated bit shift ($pin \pmod{10} \times 3$).

digital_write

Sets the state of a specified output pin to either HIGH or LOW. The implementation uses assembly to branch between the **GPSET0** and **GPCLR0** registers depending on the value parameter. By writing a bit-masked value ($1 \ll pin$) to these write-only registers, the function ensures the target pin is modified without affecting the state of other GPIO pins.

read_button

Retrieves the current logical level of a button connected to the specified GPIO pin. It accesses the **GPLEV0** register (offset 52) to read the actual voltage levels of the pins. The function applies a bitwise AND operation with a mask ($1 \ll button$) to isolate the target pin's state and returns a non-zero value if the pin is HIGH.

4.0.2 Language Distribution and Implementation

The implementation utilizes a hybrid approach, leveraging the strengths of both C and ARM Assembly. The following table summarizes the distribution of responsibilities within these hardware-accessing functions:

C Language Responsibilities

C is utilized for **interface management** and **high-level control flow**. It defines the function signatures, manages the local variable scope (such as the level variable in read_button), and handles the final return values. It also facilitates the passing of hardware constants (e.g., GPIO.GPFSEL0) into the assembly blocks.

ARM Assembler Responsibilities

Assembler is used for **direct hardware interaction**. This includes:

- **Register Math:** Calculating bit offsets ($pin \times 3$) using LSL and ADD.
- **Atomic Operations:** Using STR and LDR to ensure precise memory-mapped I/O access.
- **Bitwise Logic:** Executing BIC (Bit Clear) and ORR (Bitwise OR) to modify specific register bits without disturbing adjacent configuration.
- **Branching:** Handling conditional logic for pin-to-register mapping (e.g., switching between FSEL0, FSEL1, and FSEL2).

5 Subroutine Specification: Hamming Function

The following section outlines the design of the Hamming subroutine, focusing on its label naming and the register-based interface used for data passing

To ensure seamless integration between the C-based driver program and the low-level hardware acceleration, the Hamming function is implemented as a leaf sub-routine strictly adhering to the **ARM Architecture Procedure Call Standard (AAPCS)**. This section outlines the functional signature and the register-level protocol used for data exchange.

- The first four 32-bit parameters of the subroutine must be passed through R0 to R3. If the number of parameters exceeds four, a Stack must be used.
- The return value of a subroutine is usually placed in R0
- If the callee saves the register are needed internally in the subroutine, their contents must be pushed onto the stack (pushed) before modification and popped off the stack (Pop) before returning

hamming()

```
int hamming(const int *x, const int *y, int
seqLen)
```

Parameters

[in] x The pointer to a int sequence, not changable.

[in] **y** The pointer to another int sequence, not changable.
[in] **seqlen** A int indicate the length of the sequence.

Returns

[out] the number of positions where the two arrays differ

There are two labels: *loop*, *done*. We first check whether the sequence length is zero. If not we jump into loop. We load the value of two pointer and compare. If different then hamming distance plus one. Then we minus 1 on seqlen, load the next int of the two sequences. Once the seqlen is 0 then we are done.

In done, we move the result in r0 as return value. Then we pop to recover the register. In the end we leave the subroutine.

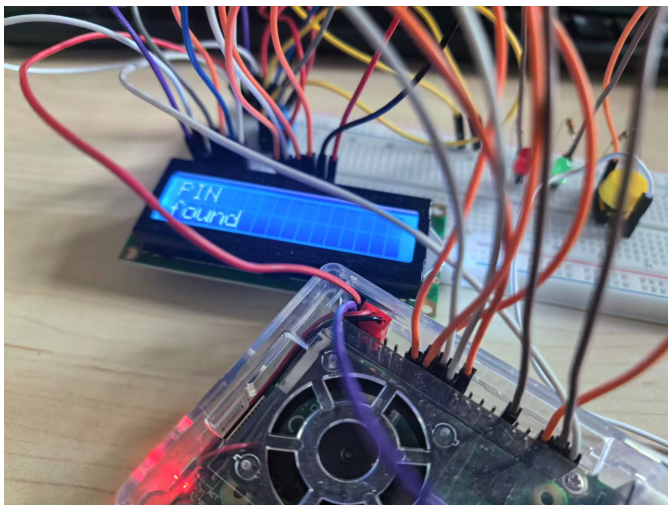
6 Implementation Analysis and Results

This section contains two parts. We will display the example output of the program, and then we shall discuss performance-relevant design decisions and implications on resource consumption.

6.1 Example Outputs

We have implemented the advanced Hamming search. Thus, we use this better version. We shall use several params which a detailed discussion can be found in Table 3. We use `-e` to make an exhaustive search so we can point out the size of the search space. Then we manually set the sequence length and digit upper bound to 6 and 12. This is quite fit as it's quite big for the simple version but quite small for the advanced version.

In order to analyze the result, we don't run the whole program but set the secret code and the input code. Below shows the terminal output of the LCD.



Terminal Output

```
raspberrypi@raspberrypi: $ sudo ./cw2 -e -n 6
-m 12 -s 123123 -r 111123
Raspberry Pi configuration: red LED: 5; green
LED: 26; button: 19
Raspberry Pi LCD driver for a 16x2 display
(4-bit wiring)
Press ENTER to continue:
-----
>> Version 5: arbitrary number of dimensions
with 12 digits and 6 sequence length
>> HAMM_ASM version: Hamming distance in ARM
Assembler
-----
Runtime: 0.036185 secs
Sequence found
Exhaustive search finished for 12 digits and 6
seqlen (expect 2985984): 2160 attempts (found
at 735 i.e. 0.02 %), 2160 submits
Secret sequence was: Contents of the sequence
(of length 6): 1 2 3 1 2 3
```

6.2 Design Trade-offs and Resource Consumption

In this subsection, we will first discuss in performance layer. We implement a brute-force search, and we have two different pruning tactics. Thus, we will do a time complexity analysis. We will discuss the submit space and search space.

6.2.1 Performance-relevant Design Decisions

We assume the sequence length to be n , and the number of digits is d , while the hamming distance is m . To represent the submission time, we set it as $O(f(n))$.

We have n position to choose and each has d possible. So the search space should be

$$d \times d \times d \cdots = d^n \quad (1)$$

We first try to build a data structure called a search space with search operations. But this is impossible as it shall consume unacceptable arrays, and we don't need to store anything.

If without hamming distance simply submit all the sequence the whole time complexity shall be

$$O(f(n) \times d^n) \quad (2)$$

But if we have hamming distance this works better. But here we want to see ignore the submit time first as when n and d increase the loop time can't be ignored. In advanced search

$$\text{attempts} = \binom{n}{m} \cdot d^m \quad (3)$$

This is much smaller than d^n and save a lot of time. This can be also found in output. It expects 2985984 attempts, but in the advanced search it only costs 2160 attempts.

Though the submit complexity is the same, the simple one has a huge loop which its time cost can't be ignore. So we this kind of search method is better.

6.2.2 implications on resource consumption

As we previously discussed, in Task 5, we adopted the strategy of generating sequences one by one instead of constructing a complete candidate list, thereby controlling the space complexity at $O(1)$.

We did not use polling but an **interval timer** to read in the numbers. This requires less CPU usage and also has lower memory usage.

In the final application, using inline ARM assembly code to directly control peripherals through GPIO without relying on any external libraries will significantly affect resource consumption. Firstly, due to the elimination of library functions and the intermediate layer of the operating system, the code size is usually smaller, and memory usage (including Flash and RAM) can be reduced to the minimum, which is very beneficial for resource-constrained embedded systems. Secondly, the execution of assembly instructions is highly controllable. The time delay and instruction cycle of each GPIO operation can be precisely calculated, thereby avoiding the additional branches, function call overhead, and context save recovery that may be introduced by library functions. Therefore, CPU utilization is more efficient in real-time response scenarios.

7 Conclusion

We have gradually finish all the task. First, we implemented direct communication with the hardware using inline coding. Subsequently, we utilized provided helper functions to display surnames on the LCD. We then enabled input reading from the buttons, which involved implementing interval timers and ensuring stable debouncing for accurate input capture.

Following this, we authored the Hamming function and the core password-cracking logic directly in Assembly. To support an arbitrary number of dimensions, we developed a faster and more efficient search algorithm. Finally, we compiled this report to document the critical components of the mission.

In this coursework, we developed a comprehensive understanding of the full stack, ranging from hardware interfacing and assembly language to high-level software logic. Initially, we studied the hardware specifications to comprehend the communication protocols between components. We mastered the use of **inline coding** for direct hardware control and adhered to **AAPCS** (ARM Architecture Procedure Call Standard) for **sub-routine** implementation. A significant portion of our development was dedicated to optimizing **two nested loops**, which substantially enhanced our algorithmic thinking. Our technical challenges and the complete workflow are documented [here](#). You can also locate there in github repo's Developing section

References